

DA3408 AI Operations, Module 1 Assignment

Question 1: Technical Debt Diagnosis

The startup's three problems each look like a different kind of hidden technical debt from the Sculley et al. paper we did in Lecture 1. I have taken the six categories to be boundary erosion, data dependencies, feedback loops, ML-system anti-patterns, configuration debt, and changes in the external world, and matched each case to one of them below.

Part 1: Which category explains each case

(a) Changing the rounding of “estimated delivery time” broke “favorite restaurants”

This is boundary erosion, and within that it is the entanglement problem, i.e. the CACE principle (Changing Anything Changes Everything).

The model does not keep its features in separate boxes. All the weights are learned together, so the prediction depends on the whole input distribution at once. Changing the rounding logic changes the distribution of the delivery-time feature, which changes what the optimal fit looks like, and when the model is retrained every other feature gets re-weighted as a side effect. The “favorite restaurants” feature has no interface or encapsulation protecting it, so there is nothing to stop the damage spreading. That is exactly what CACE says will happen: no input is really independent, so touching one feature (or a hyperparameter, or a preprocessing step, or the sampling method) moves everything else too. The tell here is that the size of the effect was much bigger than the size of the code change, which is what makes it entanglement debt rather than a normal bug.

(b) A marketing dashboard reading the model's output table without anyone knowing

This is also boundary erosion, but the specific pattern is undeclared consumers, sometimes called visibility debt.

The output table is being read by a team the ML team has never heard of, and there is no declared interface or access control in between. This is expensive for two reasons. First, the cost is invisible. The ML team cannot judge the impact of any change (a schema change, re-scoring the table, swapping the model, even fixing a bug) because it does not know who is depending on the output. The dashboard just quietly breaks, and it shows up later as a confusing complaint from the business side instead of as a failing test. Second, an undeclared consumer couples the model to a part of the stack it was never designed for, and it can quietly turn into a hidden feedback loop. If the campaign that the dashboard drives changes which restaurants users get shown and click on, those clicks come back into the training data, and the model ends up influencing its own future inputs through a path nobody has written down. So the main debt is the undeclared consumer, and the possible feedback loop is a knock-on effect of it.

(c) Training pipeline made of 14 undocumented shell scripts with no orchestration

This falls under ML-system anti-patterns, specifically a pipeline jungle (with the usual glue code sitting around it).

Pipeline jungles are described in the paper as something that grows by accident: every new data source or quick fix gets another script bolted on, until data preparation is a tangle of scrapes, joins and intermediate files. That is what 14 undocumented scripts with no orchestration tool looks like. The practical consequences are that the correct order of the steps only exists in somebody's head, failures show up late and are hard to trace back to a step, intermediate files cannot be regenerated reliably, and any end-to-end test means paying for a full re-run. Very

little of that system is actual ML code, the rest is plumbing, which is the anti-pattern. It also causes reproducibility problems later on, because for an old model you can no longer say which data version and which sequence of steps produced it.

Part 2: A mitigation for one of them

I have picked (c), the pipeline jungle.

My proposal is to throw away the 14 shell scripts and rewrite the pipeline as a DVC pipeline defined in `dvc.yaml`, with MLflow tracking each training run and the environment pinned in `environment.yml`. In more detail:

- Write the steps down as a DAG. The 14 scripts collapse into a few clear stages (ingest, clean, featurize, train, evaluate), and each stage declares its command, its input `deps` and its `outs` in `dvc.yaml`. The dependency order stops being tribal knowledge and becomes a file in Git that `dvc dag` can draw.
- Use `dvc repro` instead of re-running everything. DVC hashes the dependencies and only re-runs the stages whose inputs actually changed, so reproducing the pipeline is cheap and repeatable rather than a manual 14-step ritual.
- Version the data along with the code. `dvc add` and `dvc push` put the datasets and intermediate files on a remote (S3 or SSH), and the small `.dvc` files get committed to Git in the same commit as the code. Then any old model can be rebuilt with `git checkout <commit>` followed by `dvc checkout`.
- Move the hyperparameters and paths into `params.yaml` instead of hardcoding them in the scripts, which also cleans up the configuration debt that these pipelines collect.
- Log every run. The train stage calls `mlflow.log_param` and `mlflow.log_metric`, and also logs the seed, a `git_commit` tag and the model artifact, so the runs can be compared and traced afterwards.

A cut-down version of what the `dvc.yaml` would look like:

```
stages:
  featurize:
    cmd: python src/featurize.py data/raw data/features
    deps: [src/featurize.py, data/raw]
    params: [featurize.max_features]
    outs: [data/features]
  train:
    cmd: python src/train.py data/features models/model.pkl
    deps: [src/train.py, data/features]
    params: [train.lr, train.batch_size, train.seed]
    outs: [models/model.pkl]
    metrics: [metrics.json]
```

This fixes the actual problem, which was that the execution order was undocumented, the runs were not reproducible, and everything had to be run manually from start to finish. The `dvc.yaml` file makes the order explicit and self-documenting, caching makes partial re-runs safe, data versioning makes old runs recoverable, and MLflow ties each result back to a specific commit, data version and set of parameters. If the team later needs proper scheduling with retries and alerts, the same DAG can be moved onto something like Airflow, but DVC on its own is already enough to get rid of the jungle.